THESIS DEFENSE PREPARATION

SLM Pipeline In Depth

Every box of the adviser's pipeline explained in depth, and the code of every file explained function by function, in plain language with real values from the trained models.

Two-way ASL ↔ English Small Language Model
Prepared 27 September 2026 from the code in `slm_training_v2`
Companion to the SLM Code Study Guide · Written with Claude (AI assistant)

Hearing → Deaf: Voice Recording → Speech to text → Normalization → Reduction → Tokenize → Embedding → Contextualize → Space Restriction → Active Context → Transcribed Words → Display

Deaf → hearing: Transcribed Words → Tokenize → Embedding → Active Context → Sentence Completion → Display

Contents

How to use this guide

Part A: The basic ideas

- A1. The assembly line
- A2. Computers only understand numbers
- A3. Trained models vs. rules
- A4. Tensors and shapes
- A5. Python you will see in the code

Part B: The pipeline, box by box

- Line 1, Box 1: Voice Recording (rule; paused)
- Line 1, Box 2: Speech to text (pre-trained model, not ours; paused)
- Line 1, Box 3: Normalization (rule)
- Line 1, Box 4: Reduction (rule)
- Line 1, Box 5: Tokenize (rule)
- Line 1, Box 6: Embedding (trained model: the word picker, part 1)
- Line 1, Box 7: Contextualize (trained model: the word picker, part 2)
- Line 1, Box 8: Space Restriction (both: model scores + rules)
- Line 1, Box 9: Active Context (rule: conversation memory)
- Line 1, Boxes 10 and 11: Transcribed Words and Display (rules)
- Line 2, Box 1: Transcribed Words (rule)
- Line 2, Boxes 2 and 3: Tokenize and Embedding (Embedding is trained: TinyLlama part 1)
- Line 2, Box 4: Active Context (rule: the same notebook)
- Line 2, Box 5: Sentence Completion (both: TinyLlama + two rules)
- Line 2, Box 6: Display

Part C: Every file's code explained

- C1. vocab_pool.py: the lookup table
- C2. tokenizer.py: words ↔ numbers
- C3. onnx_config.py: synonyms and input checks
- C4. generate_dataset_v4.py: the training-data generator
- C5. make_labels.py: keep/drop labels for the word picker
- C6. encoder_distilbert.py: the encoder layers
- C7. decoder_tinyllama.py: the Sentence Completion model
- C8. word_picker.py: the word picker and the Space Restriction rules
- C9. train_tinyllama.py: training Sentence Completion
- C10. train_picker.py: training the word picker
- C11. train_distilbert.py: training Restructure (not in the pipeline now)
- C12. slm_pipeline.py: the adviser's pipeline
- C13. speech_input.py: microphone and speech-to-text
- C14. full_demo.py: the live demo with every box traced
- C15. two_way_demo.py: the compact demo
- C16. Conversation_test.py: scripted conversations
- C17. compare.py: Reduce accuracy on held-out sentences

Part D: Glossary

How to use this guide

This guide explains your SLM in depth, in two ways:

- **Part B: the pipeline, box by box.** Every box in your adviser's diagram, in order: what it does, an everyday comparison, where it is in the code, the code itself explained line by line, the real values it produced, why it's built that way, and its limits.
- **Part C: every file's code.** All 17 files in `slm_training_v2`, explained function by function.

Part A gives the few basic ideas you need first. Part D is a glossary of every technical word.

Everything here is based on the actual code, and every example value was produced by your trained models (`trained_picker.pt` and `trained_tinyllama.pt`). File names with a line number, like `slm_pipeline.py` line 240, point to the exact place in the code.

The conversation we follow all the way through:

Turn	Who	Input	Screen output
1	Hearing person	says "Where is your mother?"	Deaf person sees `where mother`
2	Deaf person	signs `home`	Hearing person sees `mother is home`

Part A: The basic ideas

A1. The assembly line

Think of the SLM as a factory assembly line. Each box in the diagram is one **station**: it takes something in, changes it, and passes it on. There are two lines:

- **Line 1, hearing → Deaf** (single solid line): Voice Recording → Speech to text → Normalization → Reduction → Tokenize → Embedding → Contextualize → Space Restriction → Active Context → Transcribed Words → Display
- **Line 2, Deaf → hearing** (solid line with broken lines): Transcribed Words → Tokenize → Embedding → Active Context → Sentence Completion → Display

In the code, every station is one method of the class `SLMPipeline` in `slm_pipeline.py`. The whole line is `non_deaf_to_deaf()` (Line 1) or `deaf_to_non_deaf()` (Line 2).

A2. Computers only understand numbers

A neural network can't read the word "mother". It can only do arithmetic. So every word must first become an **ID number** (the Tokenize box), then a **list of 64 numbers** (the Embedding box). Those 64 numbers are what the network actually works with.

A3. Trained models vs. rules

The boxes are built from two kinds of thing:

- **Trained models** learned what to do from examples. There are two in the pipeline:
  - the **word picker** (Line 1): 75,219 numbers ("parameters")
  - **TinyLlama** (Line 2): 132,800 parameters

"Training" means showing the model thousands of examples, measuring how wrong its answer is, and nudging all its numbers a little so it's less wrong next time. That's repeated for 100–150 rounds ("epochs").

- **Rules** are instructions a person wrote by hand, like "remove punctuation" or "only signs may be shown".

Every box in Part B is labelled **trained model**, **rule**, or **both**.

A4. Tensors and shapes

A **tensor** is PyTorch's name for a box of numbers with a fixed layout. Its **shape** says how many numbers there are in each direction:

- `(1, 4)`: 1 sentence, 4 word IDs
- `(1, 4, 64)`: 1 sentence, 4 words, 64 numbers per word
- `(1, 4, 17)`: 1 sentence, 4 words, 17 scores per word

When a box's explanation mentions a shape, it's telling you how much data flows through that station.

A5. Python you will see in the code

Code	Meaning
`def name(self, x):`	defines a function (a **method** when it's inside a class). `self` means "this pipeline object", so methods can reach its models and memory.
`class SLMPipeline:`	a **class** bundles data (models, memory) with the functions that use it
`[w for w in words if w in TABLE]`	a **list comprehension**: a one-line loop that builds a new list, keeping the words that pass the test
`{"mom": "mother"}`	a **dictionary**: look up a key, get a value
`d.get(w, w)`	look up `w`; if it isn't there, just use `w` itself
`set(...)`	a collection with no repeats and very fast "is it in here?" checks
`for i in range(n):`	repeat for `i` = 0, 1, …, n−1
`with torch.no_grad():`	"we are only using the model, not training it". Faster, and uses less memory.
`x.shape`	the shape of a tensor
`f"{who} is {st}"`	an **f-string**: fills in the values of `who` and `st`

Part B: The pipeline, box by box

Line 1, Box 1: Voice Recording (rule; paused)

What it does. Records the hearing person's voice from the microphone.

Everyday comparison. Pressing record on your phone's voice recorder for 5 seconds.

Where. `slm_pipeline.py` line 232 `voice_recording()` calls `speech_input.py` line 45 `record()`.

```
def record(seconds=RECORD_SECONDS):
    try:
        import sounddevice as sd
    except ImportError:
        raise SpeechUnavailable(
            "Microphone support is not installed. Run: pip install sounddevice")
    frames = sd.rec(int(seconds * SAMPLE_RATE), samplerate=SAMPLE_RATE,
                    channels=1, dtype="int16")
    sd.wait()
    return frames.tobytes()
```

Line by line.

- `import sounddevice as sd` loads the microphone library. It sits *inside* the function, so the rest of the SLM still works on a computer where the library isn't installed.
- `except ImportError: raise SpeechUnavailable(...)`: if the library is missing, stop with a clear message saying how to install it.
- `sd.rec(...)` starts recording. `seconds * SAMPLE_RATE` = 5 × 16,000 = 80,000 tiny sound measurements ("samples"). `channels=1` means mono (one microphone), and `dtype="int16"` means each sample is a whole number.
- `sd.wait()` waits until the 5 seconds are over.
- `frames.tobytes()` hands the raw sound, as bytes, to the next box.

Input → output. Nothing goes in; 5 seconds of raw audio comes out.

Why this way. 16,000 samples per second is the rate the speech model (Box 2) was trained on.

Limits. The recording always lasts exactly 5 seconds; it doesn't stop when the person stops talking. The box is paused until your group decides about speech-to-text. Until then you type the sentence, and the trace shows "skipped: typed input".

Line 1, Box 2: Speech to text (pre-trained model, not ours; paused)

What it does. Listens to the audio and writes down the words.

Everyday comparison. The automatic captions on a YouTube video.

Where. `slm_pipeline.py` line 236 `speech_to_text()` calls `speech_input.py` line 59 `transcribe()`.

```
if _model is None:
    SetLogLevel(-1)
    _model = Model(MODEL_DIR)
recognizer = KaldiRecognizer(_model, SAMPLE_RATE)
texts = []
chunk = 8000
for start in range(0, len(audio), chunk):
    if recognizer.AcceptWaveform(audio[start:start + chunk]):
        texts.append(json.loads(recognizer.Result()).get("text", ""))
texts.append(json.loads(recognizer.FinalResult()).get("text", ""))
return " ".join(t for t in texts if t).strip()
```

Line by line.

- `if _model is None:` loads the Vosk model only the first time. Loading takes a moment, so it's done once and kept.
- `SetLogLevel(-1)` silences Vosk's technical messages.
- `Model(MODEL_DIR)` loads the pre-trained English model from `models/vosk-model-small-en-us-0.15` (about 40 MB). Its makers trained it on many hours of English speech; we use it as it is.
- `KaldiRecognizer(...)` creates a "listener" that expects 16,000 samples per second.
- The `for` loop feeds the audio in pieces of 8,000 bytes (a quarter of a second each). When the listener detects a pause, `AcceptWaveform` returns True and the finished phrase is collected with `Result()`.
- `FinalResult()` collects whatever was still being heard at the end.
- The last line joins all the phrases into one transcript.

Input → output. Raw audio goes in; text such as `"where is your mother"` comes out.

Why this way. Vosk is free, works **offline** (no internet needed at the school), and is small enough for a Raspberry Pi 5.

Limits. Untested until Vosk and `sounddevice` are installed. An optional `vocabulary` setting can make Vosk choose only among your lookup-table words; it's off, because the adviser's design lets Reduction do that job.

Line 1, Box 3: Normalization (rule)

What it does. Cleans the text so it looks exactly like the sentences the word picker practised on.

Everyday comparison. A teacher tidying up an answer before marking it: removing stray punctuation, writing out short forms, and replacing nicknames with the proper name.

Where. `slm_pipeline.py` line 240 `normalization()`.

```
text = normalize_input(text).lower().replace("’", "'")
text = re.sub(r"[^\w\s']", " ", text)
words, changes = [], []
for raw in text.split():
    w = raw.strip("'")
    if not w:
        continue
    if w in CONTRACTIONS:
        parts = CONTRACTIONS[w].split()
    elif w.endswith("'s") and known_form(w[:-2]) in LOOKUP_TABLE:
        parts = [known_form(w[:-2])]          # "dad's" -> "father"
    elif w in GLOSS_MAP and w not in picker_training_words():
        parts = [GLOSS_MAP[w]]
    else:
        parts = [w.replace("'", "")]
    if parts != [w]:
        changes.append(f"{w} -> {' '.join(parts)}")
    words.extend(parts)
return words, changes
```

Line by line.

1. `normalize_input(text)` (from `onnx_config.py`) checks that the input is text, or joins a list of words into text. `.lower()` makes everything lowercase. `.replace(...)` turns a curly apostrophe (’) into a straight one (').
2. `re.sub(r"[^\w\s']", " ", text)` is a **regular expression** meaning: "replace every character that is **not** a letter or digit (`\w`), a space (`\s`) or an apostrophe with a space." This removes `? ! . ,` and so on.
3. `words, changes = [], []` starts two empty lists: the cleaned words, and a record of what was changed (shown in the demo).
4. `for raw in text.split():` handles one word at a time. `raw.strip("'")` removes apostrophes at the ends of a word (like quote marks).
5. **Rule A:** `if w in CONTRACTIONS:` means the word is a short form. `CONTRACTIONS` is a dictionary near the top of the file: `"don't" → "do not"`, `"i'm" → "i am"`, `"never" → "not"` and more. `.split()` turns `"do not"` into two words.
6. **Rule B:** `elif w.endswith("'s") ...` means a possessive. `w[:-2]` is the word without its last two characters (`'s`). `known_form()` gives the form the picker learned: `father's` → `father`, `dad's` → `father`, `mommy's` → `mother`.
7. **Rule C:** `elif w in GLOSS_MAP and w not in picker_training_words():` means a synonym the picker never saw in training is replaced by the word it knows. `GLOSS_MAP` (in `onnx_config.py`) maps `mommy → mother`, `clothes → laundry`, `mom → mother`, `dad → father`, `house → home`. `picker_training_words()` reads every word used in the picker's training sentences (`labels_train.jsonl`).
8. **Rule D:** `else:` keeps the word as spoken (removing any apostrophe left inside it). This is deliberate: words the picker **was** trained on (`okay`, `driving`) must stay in the form it learned.
9. `if parts != [w]:` records any change, like `mom -> mother`, so the demo can show it.
10. `words.extend(parts)` adds the resulting word or words to the list.

Input → output (real).

Spoken	After Normalization	Changes
`Where is your mother?`	`where is your mother`	(the `?` removed)
`I don't know where he is.`	`i do not know where he is`	`don't -> do not`
`Is your mom at home?`	`is your mother at home`	`mom -> mother`

Spoken	After Normalization	Changes
`Where is my mommy's house?`	`where is my mother home`	`mommy's -> mother; house -> home`

Why this way. A model can only handle a word well if it saw that word during training. Before this box existed, `Where is mother?` gave just `where` (the `?` broke the match), and `Is your mom at home?` gave just `home` (`mom` was never trained).

Limits. The contraction list is fixed: a short form that isn't in it (like `shouldn't`) is left as it is, and Reduction will then remove it.

Line 1, Box 4: Reduction (rule)

What it does. Removes every word that isn't in the lookup table. This happens **before** tokenizing, as your adviser specified.

Everyday comparison. A security guard with a guest list of 88 names. Anyone not on the list doesn't get in.

Where. `slm_pipeline.py` line 269 `reduction()`.

```
kept = [w for w in words if w in LOOKUP_TABLE]
removed = [w for w in words if w not in LOOKUP_TABLE]
return kept, removed
```

Line by line.

- `kept = [...]` means "go through every word; keep it **if** it's in `LOOKUP_TABLE`." `LOOKUP_TABLE = set(ALL_VOCAB)` is built at the top of the file from `vocab_pool.py`: 88 words (the 40 signs plus 48 other words).
- `removed = [...]` does the same check the other way round, collecting what was thrown out so the demo can show it.
- The pipeline also cuts the list to 16 words (`MAX_WORDS`), because the word picker's position table only has 16 rows.

Input → output (real). `where did your mother park the car` → kept `where your mother the`, removed `did, park, car`.

Why 88 words and not the 40 signs? Some non-sign words carry meaning the model needs. `is` at the start is how it recognises `is father okay` as a question, which gives `how father`. Narrowing down to the 40 signs happens in Box 8, after the model has understood the sentence. Every word in the picker's training sentences is in the lookup table (checked on all 827 labelled rows), so Reduction never removes anything the model learned from.

Line 1, Box 5: Tokenize (rule)

What it does. Turns each word into its ID number.

Everyday comparison. A class list where every student has a number.

Where. `slm_pipeline.py` line 338 `tokenize()`, using the table built in `tokenizer.py` lines 28–31.

```
return [self.tokenizer.token_to_id[w] for w in words]
```

The table (`tokenizer.py`):

```
all_tokens = SPECIAL_TOKENS + sorted(set(vocab_words))
self.token_to_id = {tok: i for i, tok in enumerate(all_tokens)}
```

Line by line.

- `SPECIAL_TOKENS` are four markers: `<pad>` = 0 (filler), `<bos>` = 1 (beginning of sentence), `<eos>` = 2 (end of sentence), `<unk>` = 3 (unknown word).
- `sorted(set(vocab_words))` gives the 88 words in alphabetical order, with no repeats.
- `enumerate(...)` counts 0, 1, 2, … while going through the list, so each word gets the next number. That's 92 IDs in total.
- In `tokenize()`, each word is looked up in `token_to_id`. After Reduction, every word is guaranteed to have an ID.

Input → output (real). `where is your mother` → `[85, 41, 91, 51]`.

Why this way. One ID per whole word (**word-level** tokenizing) is enough for a small, fixed vocabulary. Big models like ChatGPT cut words into pieces because they must handle any word at all.

Line 1, Box 6: Embedding (trained model: the word picker, part 1)

What it does. Turns each ID into a list of 64 numbers that describes the word **and** its place in the sentence.

Everyday comparison. Each word gets a **profile card** with 64 scores, plus a **seat-number sticker**. An ID like 51 is just a jersey number and says nothing about the word; the profile card does.

Where. `slm_pipeline.py` line 342 `embedding()`, which calls `word_picker.py` line 94 `embed()`, which runs `encoder_distilbert.py` line 31, `DistilBertEmbeddings.forward()`.

```
def forward(self, input_ids):
    seq_len = input_ids.size(1)
    position_ids = self.position_ids[:, :seq_len]
    word_emb = self.word_embeddings(input_ids)
    pos_emb = self.position_embeddings(position_ids)
    embeddings = word_emb + pos_emb
    embeddings = self.layer_norm(embeddings)
    embeddings = self.dropout(embeddings)
    return embeddings
```

Line by line.

- `seq_len = input_ids.size(1)`: how many words there are (4).
- `position_ids = ...`: the positions 0, 1, 2, 3.
- `self.word_embeddings(input_ids)`: the **word table**, 92 rows × 64 numbers. ID 51 picks row 51, which is `mother`'s card. The numbers in this table were learned during training.
- `self.position_embeddings(position_ids)`: the **position table**, 16 rows × 64 numbers, one row per position.
- `word_emb + pos_emb`: adding them makes each word's 64 numbers say both *what* it is and *where* it sits.
- `self.layer_norm(...)`: LayerNorm rescales the numbers to a steady range (like adjusting the volume).
- `self.dropout(...)`: during training this randomly switches off some numbers, so the model can't rely on any single one. While the pipeline is running it's switched off (the model is in `eval()` mode).

Input → output (real), for `mother` (first 4 of 64 numbers):

Step	Numbers
word table row 51	`[+0.06, -0.73, +1.31, +1.01, ...]`
position table row 3	`[+1.60, -0.79, -0.71, +0.58, ...]`
added	`[+1.66, -1.52, +0.61, +1.59, ...]`
after LayerNorm	`[+1.40, -1.71, +0.65, +1.48, ...]`

The whole sentence becomes a tensor of shape `(1, 4, 64)`.

Why this way. Position matters for word picking: `is` at the start of a sentence usually means a question, so the model needs to know where each word sits.

Line 1, Box 7: Contextualize (trained model: the word picker, part 2)

What it does. Lets every word "read" all the other words, so its 64 numbers describe the word **in this sentence**.

Everyday comparison. A group discussion. Each word asks the others "which of you matters to me?", listens more to the relevant ones, and updates its own notes. The discussion happens twice.

Where. `slm_pipeline.py` line 277 `contextualize()` → `word_picker.py` line 98 `contextualize()` → `encoder_distilbert.py` `TransformerBlock` (line 110) and `MultiHeadSelfAttention` (line 45).

```
def contextualize(self, x, padding_mask):
    for layer in self.layers:
        x = layer(x, key_padding_mask=padding_mask)
    return x
```

Each `layer` is a `TransformerBlock`:

```
def forward(self, x, key_padding_mask=None):
    attn_out = self.attention(self.norm1(x), key_padding_mask)
    x = x + self.dropout(attn_out)
    ffn_out = self.ffn(self.norm2(x))
    x = x + self.dropout(ffn_out)
    return x
```

And the heart of `self.attention`:

```
q = self.q_lin(x).view(B, S, self.n_heads, self.head_dim).transpose(1, 2)
k = self.k_lin(x).view(B, S, self.n_heads, self.head_dim).transpose(1, 2)
v = self.v_lin(x).view(B, S, self.n_heads, self.head_dim).transpose(1, 2)
scale = math.sqrt(self.head_dim)
scores = torch.matmul(q, k.transpose(-2, -1)) / scale
attn_weights = F.softmax(scores, dim=-1)
context = torch.matmul(attn_weights, v)
```

Line by line.

- The `for` loop runs the sentence through **2 transformer blocks**, one after another.
- In each block, `self.attention(...)` is the discussion and `self.ffn(...)` is each word thinking alone. `norm1` and `norm2` rescale first (this is called "Pre-LN").
- `x = x + ...` is a **residual connection**: the block *adds* its changes to the word's old notes instead of replacing them, like editing a draft with tracked changes. This keeps training stable.

- Inside attention, each word makes three things from its 64 numbers:
  - `q`, a **query**: "what am I looking for?"
  - `k`, a **key**: "what do I contain?"
  - `v`, a **value**: "what can I share?"

`.view(...).transpose(1, 2)` splits the 64 numbers into **4 heads of 16**, so four separate discussions happen at once, each free to focus on a different kind of relationship.

- `scores = q × k / scale`: every query is compared with every key, giving a relevance score for each pair of words (a 4 × 4 table per head). Dividing by √16 = 4 keeps the numbers moderate.
- `softmax` turns each row of scores into percentages that add up to 100%.
- `context = weights × v`: each word's new notes are a mix of everyone's values, weighted by those percentages.
- The feed-forward part (`FFN`, in `encoder_distilbert.py` line 93) expands 64 → 128 numbers, applies GELU, and shrinks back to 64.

Input → output (real). The shape stays `(1, 4, 64)`, but the numbers change. The same word gets different numbers in different sentences:

Sentence	`mother` after Contextualize
"where is your mother"	`[+1.39, -7.89, -1.07, +1.13, ...]`
"mother is sick"	`[-5.23, +3.20, -1.38, -0.27, ...]`

Why this way. Whether a word should be kept depends on the other words around it. Because this attention looks both forwards and backwards (it's "bidirectional"), every word can use the whole sentence.

Line 1, Box 8: Space Restriction (both: model scores + rules)

What it does. Decides which words become **signs**, and in what order. It narrows the 88-word space down to the 40-sign space.

Everyday comparison. A multiple-choice test for every word, then a guest filter, then a seating chart.

Where. `slm_pipeline.py` line 284 `space_restriction()` → `word_picker.py` line 105 `heads()` and line 134 `restrict_to_signs()`.

Step 1: the model's scores (`heads`).

```
tag_logits = self.tag_head(x)
keep = (~padding_mask).unsqueeze(-1).float()
pooled = (x * keep).sum(dim=1) / keep.sum(dim=1).clamp(min=1.0)
question_logits = self.question_head(pooled)
```

- `self.tag_head(x)` turns each word's 64 numbers into **17 scores**. Score 0 means "drop me"; scores 1–16 mean "keep me, in seat 1–16". Shape `(1, 4, 17)`.
- `keep` marks real words with 1 and padding with 0.
- `pooled` is the **average** of all the real words: one 64-number summary of the whole sentence.
- `self.question_head(pooled)` gives 2 scores: `[normal, add "how"]`.

Step 2: the rules (`restrict_to_signs`).

```
is_question = int(q_logits[0].argmax().item())
for i in range(n):
    w = normalize_word(words[i].lower())
    if words[i].lower() in NEGATIONS or w in NEGATIONS:
        negated = True
        continue
    if w not in SIGNS:
        continue
    if scores[i, 1:].max() > scores[i, 0]:
        keep_idx.append(i)
```

- `argmax()` picks whichever of the 2 question scores is bigger.
- `normalize_word(...)` gives the sign's name for a word (`your` → `you`, `driving` → `drive`).
- **Negation rule:** words in `NEGATIONS` (`not`, `don't`, `never`, … 14 in all) aren't shown themselves. They set `negated = True`, and the sign `no` is added later.
- **Signs-only rule:** if a word has no sign, `continue` skips it. It's dropped, whatever the model scored.
- **Keep rule:** a sign word is kept if its best keep score (`scores[i, 1:].max()`) beats its drop score (`scores[i, 0]`).

Step 3: the seating chart (Hungarian algorithm).

```
sub = scores[keep_idx][:, 1:len(keep_idx) + 1]
rows, cols = linear_sum_assignment(-sub)
for r, c in zip(rows, cols):
    labels[keep_idx[r]] = int(c) + 1
```

- `sub` is a small table: the kept words (rows) × their scores for seats 1 to k (columns).
- `linear_sum_assignment(-sub)` is SciPy's Hungarian algorithm. It finds the one-to-one word→seat match with the **highest total score**. It looks for the *lowest* total, so the scores are made negative first.
- The loop writes each word's seat number into `labels`.
- Finally, the words are sorted by seat; `no` is added at the front if a negation was heard, and `how` at the front if the question flag was set.

Input → output (real):

Word	Drop score	Best keep score	Decision
where	0.73	seat 1: 22.41	keep, seat 1
is	17.81	lower	drop (also, `is` has no sign)
your	21.57	lower	drop
mother	-5.11	seat 2: 14.22	keep, seat 2

The question scores were `[11.91, -9.16]`, so no "how" is added. The result is `where mother`, with labels `[1, 0, 0, 2]`.

Why this way. The model can only choose from words that were actually spoken; it can never invent one. On 294 sentences it never saw in training, it's right on 290 (98.6%). The Hungarian algorithm guarantees no two signs can share a seat, so no sign is ever lost.

Limits. Some lookup-table words never appeared in the training sentences (`what`, `finish`, `he`, `she`, `going`, …), so the model's scores for them are guesses. It also never learned the wording "is **your** … okay": `Is your mother okay?` gives `mother`, missing `how`. The fix is to add training sentences and retrain.

Line 1, Box 9: Active Context (rule: conversation memory)

What it does. Keeps a notebook of the conversation, and uses it to replace "he" or "she" with the person being talked about.

Everyday comparison. When your friend asks "Is she okay?", you know "she" means the mother you were just discussing.

Where. `slm_pipeline.py`, class `ActiveContext` (line 92): `resolve_pronoun()` (line 113) and `note_hearing()` (line 129).

```
def resolve_pronoun(self, spoken_words, glosses):
    if self.person is None or any(g in PEOPLE for g in glosses):
        return glosses, None
    said = set(spoken_words)
    if not any(p in said and self.person in fits
               for p, fits in PRONOUN_FITS.items()):
        return glosses, None
    i = 0
    while i < len(glosses) and (glosses[i] in QUESTIONS or glosses[i] == "no"):
        i += 1
    return glosses[:i] + [self.person] + glosses[i:], self.person
```

Line by line.

- If nobody is remembered yet, or the signs already contain a person, change nothing.
- `PRONOUN_FITS` says `he` can be father, teacher or friend, and `she` can be mother, teacher or friend. `family` is plural, so neither pronoun refers to it. The `if not any(...)` line asks: "did they say he or she, and does the remembered person fit that word?" If not, change nothing.
- The `while` loop skips past a leading question word or `no`, so the person is inserted where the training outputs put it: `where father`, `how mother`, `no where father`.

```
def note_hearing(self, spoken_words, glosses):
    words = [normalize_word(w) for w in spoken_words] + list(glosses)
    self.last_question_words = {w for w in words if w in QUESTIONS}
    self.last_was_question = bool(self.last_question_words)
    if "you" in spoken_words:
        self.addressed_to_you = True
    elif any(w in PEOPLE for w in words):
        self.addressed_to_you = False
    self._remember_person(words)
    self.turns += 1
```

Line by line. This writes in the notebook:

- **which question words** were used (`where`, `how`, ...), and whether it was a question at all;
- **whether it was asked to the Deaf person.** Only the word `you` itself counts. `your father` is about father, so it doesn't; that was the old "your" bug;
- **the last person mentioned** (`_remember_person`), and one more turn in the turn count.

Input → output (real). After Turn 1 the notebook holds: person = mother, question words = {where}, asked to the Deaf person = no. A later turn, `Is she okay?`, gives `how` from Box 8, and Active Context turns it into `how mother`.

Why this way. Real conversations drop names once they've been mentioned. It's written as rules so the behaviour is predictable and easy to explain. Training context into the model was tried earlier, and an ablation showed no benefit.

Limits. Only `he` and `she` are handled (not "they" or "him"). Only one person is remembered at a time.

Line 1, Boxes 10 and 11: Transcribed Words and Display (rules)

Where. `slm_pipeline.py` line 295 `transcribed_words()` and line 351 `display()`.

```
def transcribed_words(self, glosses):
    return " ".join(glosses)

def display(self, turn, text):
    turn.output = text
    return text
```

`" ".join(glosses)` joins the list `["where", "mother"]` into the text `where mother`. `display()` stores it as the turn's screen output. `full_demo.py` and `two_way_demo.py` then draw the "DEAF PERSON'S SCREEN" box.

Line 2, Box 1: Transcribed Words (rule)

What it does. Receives the words Veron's sign recognizer produced, and checks them.

Where. `slm_pipeline.py` line 301 `transcribed_signs()`.

```
text = normalize_input(recognized)
signs, ignored = [], []
for w in self.tokenizer._clean_and_split(text):
    s = normalize_word(w)
    if s in SIGNS:
        signs.append(s)
    else:
        ignored.append(w)
return signs, ignored
```

Line by line.

- `normalize_input` accepts either text (`"drive mother where"`) or a list (`["drive", "mother", "where"]`), so Veron's module can send either.
- `_clean_and_split` lowercases, removes punctuation, and splits into words.
- `normalize_word(w)` turns a recognizer label into the sign's name (`mama` → `mother`).
- Words that are one of the 40 signs are kept; anything else is put aside and shown in the trace.

Input → output (real).

- `home` → `home`
- `mama laundry xyz` → `mother laundry`, with `xyz` set aside

Why no Normalization or Reduction on this line? The camera can only produce sign words, so there's nothing to clean.

Line 2, Boxes 2 and 3: Tokenize and Embedding (Embedding is trained: TinyLlama part 1)

Tokenize is the same box as on Line 1: `home` → `[37]`.

Embedding uses **TinyLlama's own table**: `decoder_tinyllama.py` line 239 `embed_source()`.

```
def embed_source(self, input_ids):
    return self.src_embedding(input_ids)
```

It's a plain lookup in a learned table of 92 rows × 64 numbers. Real numbers for `home`: `[-1.17, -1.76, -0.05, -0.42, ...]`.

Important difference from Line 1: no position is added. TinyLlama doesn't know which sign came first. That's why the adviser could leave out Restructure (reordering the signs): reordering can't change what TinyLlama writes. This was tested on all 979 sign inputs, and every sentence was identical with and without it.

The cost: "mother help father" and "father help mother" look the same to TinyLlama.

Line 2, Box 4: Active Context (rule: the same notebook)

What it does. If the signs are missing *who* they're about, it adds the person from the notebook.

Everyday comparison. Someone asks "Where is your mother?" and you answer "Home." Everyone understands "*Mother* is home."

Where. `slm_pipeline.py` line 147 `fill_subject()`, plus lines 446–449 in `deaf_to_non_deaf()`.

```
if any(s in PEOPLE for s in signs):
    return signs, None, "a person was signed"
if "my" in signs:
    return signs, None, "'my': the signer is talking about themselves"
if self.person is None:
    return signs, None, "no person remembered yet"
needs_subject = (
    any(s in QUESTIONS for s in signs)
    or (any(s in STATES for s in signs)
        and not any(s in ACTIVITIES for s in signs))
    or ("where" in self.last_question_words
        and all(s in PLACES for s in signs))
)
if not needs_subject:
    return signs, None, "this turn does not need a subject"
if self.addressed_to_you:
    return signs, None, "the last question was asked to the Deaf person"
return [self.person] + signs, self.person, f"added {self.person!r} from earlier"
```

Line by line. The checks run in order. The first one that applies decides:

1. A person was signed (`mother drive`), so nothing is needed.
2. `my` was signed (`my sick`), so the signer means themself.
3. Nobody is remembered yet.
4. Does the turn **need** a subject? Yes if it has a question word (`where`), or a state (`fine`) without an activity, or a bare place (`home`) answering a *where* question.
5. If the last question was asked to the Deaf person ("How are **you**?"), don't add anyone. A reply of `fine` means *they* are fine.
6. Otherwise, put the remembered person in front.

Because the diagram places this box **after** Embedding, the person is added as numbers:

```
extra = self.embedding(self.tokenize([added]), "signs")
vectors = torch.cat([extra, vectors], dim=1)
```

This embeds the person's name and joins its 64 numbers to the front of the sign vectors (`torch.cat` means "concatenate": join side by side).

Input → output (real). `home`, after "Where is your mother?" → `added 'mother' from earlier`. The signs become `[mother, home]`.

Line 2, Box 5: Sentence Completion (both: TinyLlama + two rules)

What it does. Writes the English sentence one word at a time, then checks it.

Everyday comparison. The word suggestions on a phone keyboard, where you keep tapping the top suggestion. The difference is that TinyLlama also keeps looking at the signs while it writes.

Where. `slm_pipeline.py` line 316 `sentence_completion()` → `decoder_tinyllama.py` line 271 `generate_from_memory()` and line 243 `decode()`.

```
generated = torch.tensor([[tokenizer.bos_id]], device=device)
with torch.no_grad():
    for _ in range(max_new_tokens):
        logits = self.decode(memory, generated, src_padding_mask)
        next_token = logits[0, -1, :].argmax().item()
        if next_token == tokenizer.eos_id:
            break
        generated = torch.cat([generated,
                               torch.tensor([[next_token]], device=device)], dim=1)
return tokenizer.decode(generated[0, 1:].tolist(), skip_special_tokens=True)
```

Line by line.

- Start the sentence with only the `<bos>` ("begin") marker.
- Repeat up to 16 times:
  - `self.decode(...)` scores all 92 words for the next position.
  - `logits[0, -1, :]` takes the scores for the **last** position only, and `.argmax()` picks the highest-scoring word (this is called **greedy** decoding).
  - If it's `<eos>` ("end"), stop.
  - Otherwise add the word to the sentence and go around again.
- Finally, turn the IDs back into words, skipping the markers.

Inside `decode()`, the sentence written so far goes through 2 decoder blocks. Each block does three things:

1. **causal self-attention with RoPE:** each written word looks only at the words written *before* it (it can't peek ahead), and RoPE tells it how far apart words are;
2. **cross-attention:** the written words look at the sign vectors, which is how the signs steer the sentence;
3. **SwiGLU feed-forward:** each word thinks it over alone.

Then the **output head** turns 64 numbers into 92 scores, one per word.

The two rules afterwards (`sentence_completion`):

```
if self.context.last_was_question:
    stated = to_declarative(sentence)
    ...
missing = [s for s in signs if not covered(s, set(sentence.split()))]
if missing:
    sentence = " ".join(signs)
```

- **Answer form** (`to_declarative`, line 187): if the hearing person had just asked a question, a sentence starting with `is` is turned into a statement: `is mother home` → `mother is home`.
- **Safety net** (`covered`, line 200): every sign must appear in the sentence, either as itself, as a synonym, or as a word starting with the sign's first 4 letters (`driv` matches `driving`). If any sign is missing, the plain signs are shown instead of a fluent but wrong sentence.

Input → output (real):

Round	Written so far	TinyLlama picks	Confidence
1	(begin)	`is`	100%
2	is	`mother`	100%
3	is mother	`home`	100%
4	is mother home	(end)	100%

Answer form: `is mother home` → `mother is home`. Safety net: `mother` ✓, `home` ✓.

Limits. Greedy decoding never reconsiders a word once it's written. Only 46 of the held-out test rows check this model. The safety net checks that each sign is *present*, not that the sentence *means* the right thing.

Line 2, Box 6: Display

`display()` stores `mother is home` as the output, and the demo draws the "HEARING PERSON'S SCREEN" box.

Part C: Every file's code explained

The files are grouped the way they depend on each other: foundation → data → models → training → pipeline → demos and tests.

C1. vocab_pool.py: the lookup table

Purpose. The master word list every other file reads.

```
CONFIRMED_GESTURES = ["yes", "no", "good", "fine", ..., "drive", "laundry"]   # 40 signs
HELPER_WORDS = ["i", "you", "your", "am", "is", ..., "look"]                  # 54 words
ALL_VOCAB = sorted(set(CONFIRMED_GESTURES + HELPER_WORDS))                   # 88 words
CATEGORIES = {"mother": "Person / Family", "where": "Basic Question", ...}   # 40 entries
```

- `CONFIRMED_GESTURES`: the only words Veron's camera can produce.
- `HELPER_WORDS`: never signed. They're either grammar words TinyLlama writes (`is`, `am`), or words hearing people say that the picker must recognise so it can drop them (`the`, `very`, `okay`).
- `ALL_VOCAB`: `+` joins the lists (94 words); `set` removes the 6 that appear in both (`you, my, his, her, please, want`); `sorted` puts them in alphabetical order so the IDs never change.
- `CATEGORIES`: `slm_pipeline.py` builds `PEOPLE`, `QUESTIONS`, `STATES`, `ACTIVITIES` and `PLACES` from it for the Active Context rules.
- `if __name__ == "__main__":` runs only when you type `python vocab_pool.py`, and prints the counts.

Note. Adding a sign means adding it here **and** in `generate_dataset_v4.py`, regenerating the data, and retraining.

C2. tokenizer.py: words ↔ numbers

```
class SimpleTokenizer:
    def __init__(self, vocab_words):
        all_tokens = SPECIAL_TOKENS + sorted(set(vocab_words))
        self.token_to_id = {tok: i for i, tok in enumerate(all_tokens)}
        self.id_to_token = {i: tok for i, tok in enumerate(all_tokens)}
        self.pad_id = self.token_to_id["<pad>"]
        ...
        self.vocab_size = len(all_tokens)
```

- `__init__` runs when the tokenizer is created. It builds the two tables (word → ID, and ID → word) and remembers the IDs of the special markers and the total size (92).

```
def _clean_and_split(self, text):
    text = text.lower()
    text = re.sub(r"[^\w\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text.split() if text else []
```

- Lowercases the text, turns every punctuation mark into a space, squashes repeated spaces, and splits into words.

```
def encode(self, text, add_special_tokens=True):
    words = self._clean_and_split(text)
    ids = [self.token_to_id.get(w, self.unk_id) for w in words]
    if add_special_tokens:
        ids = [self.bos_id] + ids + [self.eos_id]
    return ids
```

- Cleans the text, then looks each word up. `.get(w, self.unk_id)` means an unknown word becomes `<unk>` (3) instead of crashing. It optionally wraps the IDs in `<bos>` … `<eos>`. Example: `encode("Where is mother?")` → `[1, 85, 41, 51, 2]`.
- `decode(ids)` does the reverse, skipping the markers.
- `save` / `load` write or read the table as a JSON file (only the self-test uses them).

C3. onnx_config.py: synonyms and input checks

```
GLOSS_MAP = {
    "mama": "mother", "papa": "father", "mom": "mother", ...,
    "driving": "drive", "eating": "eat", ...,
    "your": "you", "me": "my", "need": "want", "hi": "hello", "done": "finish",
}
```

- The single table of synonyms and word forms. Used by `make_labels.normalize_word()` (and through it, the whole pipeline) and by Normalization. `good` is deliberately absent: it's its own sign.

```
def normalize_input(text):
    if text is None:
        raise TypeError("SLM received None as input. ...")
    if isinstance(text, (list, tuple)):
        text = " ".join(str(w) for w in text)
    if not isinstance(text, str):
        raise TypeError(f"SLM received input of type {type(text).__name__}, ...")
    return text.strip()
```

- **Defensive programming:** fail early with a clear message if the input is `None` or a number. A list of words is joined into one string.
- The other names in the file (`pad_ids`, `map_glosses`, `sanitize_glosses`, `enforce_max_len`, `oov_density`, `reduce_confidence`, and the ONNX paths and lengths) are used by the on-hold Raspberry Pi files in `pi_deployment_later/`, not by the pipeline.

C4. generate_dataset_v4.py: the training-data generator

Purpose. Writes `dataset_v4_train.jsonl` and `dataset_v4_test.jsonl` from **rules over sign groups**, not hand-written sentences.

The sign groups:

```
PERSON = ["mother", "father", "teacher", "family", "friend"]
STATE = ["fine", "sad", "tired", "sick", "good", "deaf"]
ACTIVITY = ["drive", "laundry", "study", "learn", "eat"]
...
VERB = {"drive": "driving", "laundry": "doing laundry", ...}
```

- Each group lists the signs that behave the same way grammatically. `VERB` says how an activity reads inside a sentence.

Sentence Completion rules (`build_complete`):

```
for pr in PERSON:
    who = PERSON_ARTICLE[pr]
    for st in STATE:
        add([pr, st], f"{who} is {st}")
```

- A loop inside a loop covers every person with every state: 5 × 6 = 30 patterns. For example, `mother sad` → `mother is sad`. `add()` includes both sign orders for two-sign inputs. `SINGLE` gives the meaning of one sign alone (`sad` → `i am sad`).

Word picker rules (`build_reduce`):

```
pats.append(([f"{p} is {s}", f"my {p} is {s}",
              f"your {p} is {s}", f"{p} is very {s}"],
             f"{p} {s}"))
```

- Several spoken phrasings, one sign output. This forces the picker to learn the rule (drop `my`, `your`, `is`, `very`) instead of memorising sentences.

Rows, split and checks:

- `build_rows()` turns each pattern into rows like `{"task": "reduce", "input": "my mother is sad", "output": "mother sad", "pattern": "reduce:mother sad"}`.
- `split_by_phrasing()` groups the rows by meaning and holds out up to 2 phrasings per meaning for testing, keeping at least 2 for training. A single-sign input is never held out.
- `check_task_shapes()` makes sure Sentence Completion inputs are signs only, and word-picker outputs are signs only.
- `check_no_contradictions()` makes sure the same input never has two different answers. If either check fails, the script stops before writing anything.

Real sizes: 2,409 training rows (935 Complete, 533 Reduce, 941 Restructure) and 342 test rows (46, 294, 2).

C5. make_labels.py: keep/drop labels for the word picker

Purpose. Turns each Reduce row (sentence → signs) into **one label per word** (0 = drop, 1–16 = seat), plus a question flag.

```
def derive_labels(input_text, output_words):
    in_words = input_text.split()
    labels = [0] * len(in_words)
    used_input = [False] * len(in_words)
    unmatched = []
    for out_pos, ow in enumerate(output_words, start=1):
        candidates = [i for i, iw in enumerate(in_words)
                      if not used_input[i] and (iw == ow or normalize_word(iw) == ow)]
        if not candidates:
            unmatched.append(ow)
            continue
        found = candidates[-1]
        used_input[found] = True
        labels[found] = out_pos
    return labels, unmatched
```

- Start with every word dropped (`[0, 0, 0, 0]`).
- For each output sign, in order (seat 1, seat 2, ...), find the input words that match it, exactly or after `normalize_word`. `candidates[-1]` takes the **last** match: in `tell me my name`, it's the possessive `my`, not `me`, that carries the meaning.
- Mark that word as used and give it the seat number. An output word that isn't in the input goes to `unmatched`.
- Example: `where is your mother` → `where mother` gives `[1, 0, 0, 2]`.

```
def label_row(input_text, output_text):
    out_words = output_text.split()
    labels, unmatched = derive_labels(input_text, out_words)
    if not unmatched:
        return labels, 0
    if out_words and out_words[0] == QUESTION_MARKER and set(unmatched) == {QUESTION_MARKER}:
        labels, unmatched = derive_labels(input_text, out_words[1:])
        if not unmatched:
            return labels, 1
    return None
```

- First try plain labelling. If the **only** missing word is a leading `how`, label the rest and set the question flag to 1 (`is father okay` → `how father`). Otherwise the row can't be expressed this way, and is skipped.
- `rebuild()` and `verify()` rebuild every answer from its labels and compare it with the real one, which catches labelling mistakes before any training.
- `main()` does this for the train and test splits and writes `labels_train.jsonl` (533 rows) and `labels_test.jsonl` (294 rows).

C6. encoder_distilbert.py: the encoder layers

Purpose. Defines the DistilBERT-style layers. Today they're the **body of the word picker**: Embedding (`DistilBertEmbeddings`) and Contextualize (`TransformerBlock`). The file also defines the full Restructure model, which isn't in the pipeline now.

Class	What it does	Explained in
`DistilBertEmbeddings`	word table + position table, then LayerNorm	Part B, Embedding
`MultiHeadSelfAttention`	4-head attention in both directions; padding gets 0% attention	Part B, Contextualize

Class	What it does	Explained in
`FFN`	64 → 128 → 64, with GELU	Part B, Contextualize
`TransformerBlock`	attention + FFN, each with a residual	Part B, Contextualize
`DistilBertForRestructure`	the Restructure model: embeddings → 2 blocks → 16 position scores per word	not used by the pipeline

Padding mask, one extra detail:

```
if key_padding_mask is not None:
    scores = scores.masked_fill(key_padding_mask.unsqueeze(1).unsqueeze(2), float("-inf"))
```

When sentences of different lengths are trained together, short ones are filled with `<pad>`. Setting their scores to minus infinity makes softmax give them exactly 0% attention, so padding never affects real words.

The body's parameters: word table 5,888 + position table 1,024 + LayerNorm 128 + 2 blocks × 33,472 = **73,984**.

C7. decoder_tinyllama.py: the Sentence Completion model

Name	What it does
`RMSNorm`	rescales numbers by their root-mean-square (Llama's simpler LayerNorm)
`precompute_rope_freqs`, `apply_rope`	RoPE: rotates pairs of numbers by an angle that grows with position
`SwiGLU`	gated feed-forward: `down(silu(gate(x)) * up(x))`
`TinyLlamaCausalAttention`	attention where a word only sees earlier words
`CrossAttention`	the sentence being written reads the sign vectors
`TinyLlamaBlock`	self-attention → cross-attention → SwiGLU, each with RMSNorm and a residual
`TinyLlamaForGeneration`	the full model

RMSNorm:

```
rms = x.pow(2).mean(-1, keepdim=True).add(self.eps).sqrt()
return x / rms * self.weight
```

Square the numbers, average them, and take the square root. Divide by that, then multiply by learned weights. It keeps the numbers at a steady size.

RoPE:

```
x_r[..., 0] * cos - x_r[..., 1] * sin,
x_r[..., 0] * sin + x_r[..., 1] * cos,
```

This is the standard formula for rotating a point on a circle. Each pair of numbers in the query and key is rotated by an angle that depends on the word's position, so the attention between two words depends on how far apart they are.

The causal mask:

```
causal = torch.triu(torch.ones(max_len, max_len), diagonal=1).bool()
```

`triu(..., diagonal=1)` marks every "future" position, and those get 0% attention. While writing word 3, the model can't see word 4.

The methods the pipeline uses:

- `embed_source(ids)`: the Embedding box (Line 2).
- `decode(memory, target_ids)`: scores the next word; used by `generate_from_memory()` (Sentence Completion).
- `forward()` = `decode(embed_source(ids), ...)`, used in training. `generate()` = `generate_from_memory(embed_source(ids))`.

These produce exactly the same outputs as before the pipeline restructure (verified on all 979 inputs).

Parameters: two word tables 11,776 + 2 blocks 115,072 + final norm and output head 5,952 = **132,800**.

C8. word_picker.py: the word picker and the Space Restriction rules

```
class WordPicker(nn.Module):
    def __init__(self, vocab_size, hidden_size=64, n_heads=4, n_layers=2,
                 ffn_size=128, max_len=16, pad_id=0, dropout=0.1):
        super().__init__()
        self.embeddings = DistilBertEmbeddings(vocab_size, hidden_size, max_len, pad_id, dropout)
        self.layers = nn.ModuleList([TransformerBlock(hidden_size, n_heads, ffn_size, dropout)
                                     for _ in range(n_layers)])
        self.tag_head = nn.Linear(hidden_size, max_len + 1)
        self.question_head = nn.Linear(hidden_size, 2)
```

- It's built from the encoder's parts: embeddings, 2 transformer blocks, then **two heads**. `nn.Linear(64, 17)` turns 64 numbers into 17 scores. `max_len + 1` = 16 seats + 1 "drop" choice.
- `embed()`, `contextualize()` and `heads()` are the three pipeline boxes. `forward()` simply runs them in order.
- `restrict_to_signs()` holds the Space Restriction rules (Part B, Box 8).
- `picker_predict()` runs the whole Reduce step in one call (clean the words → IDs → model → rules). It's used by `compare.py`. It cleans words the same way the tokenizer does and cuts input at 16 words.
- **Parameters:** body 73,984 + tag head 1,105 + question head 130 = **75,219**.

C9. train_tinyllama.py: training Sentence Completion

```
def load_generation_examples(path, tasks=("complete",)):
```

This keeps only the Complete rows: 935 of them. (The model was once trained on Reduce rows as well, and it mixed up the two jobs.)

```
src = tokenizer.encode(input_text, add_special_tokens=False)
tgt = tokenizer.encode(output_text, add_special_tokens=True)
```

The signs need no markers; the sentence gets `<bos>` … `<eos>`, so the model learns where to start and when to stop.

```
tgt_input  = tgt_ids[:, :-1]
tgt_labels = tgt_ids[:, 1:]
optimizer.zero_grad()
logits = model(src_ids, tgt_input)
loss = criterion(logits.reshape(-1, logits.size(-1)), tgt_labels.reshape(-1))
loss.backward()
optimizer.step()
```

- **Teacher forcing:** for `is mother home`, the model is given `<bos> is mother home` and must predict `is mother home <eos>`, one position at a time.
- The 5 training steps:
  - clear the old corrections
  - predict
  - measure how wrong the prediction is (cross-entropy **loss**)
  - work out how to change every number (**backpropagation**)
  - change them a little (the **AdamW** optimizer, learning rate 0.001)
- This repeats for 150 epochs, in batches of 8.
- `torch.save({...})` stores the learned numbers **and** the settings (`"variant": "tinyllama"`, the sizes), so the pipeline can rebuild the model.
- `set_seed()` fixes all randomness to 42, so retraining gives the same model.

C10. train_picker.py: training the word picker

```
tag_logits, q_logits = model(input_ids)
tag_loss = tag_criterion(tag_logits.reshape(-1, tag_logits.size(-1)), labels.reshape(-1))
q_loss = q_criterion(q_logits, questions)
loss = tag_loss + q_loss
```

- It reads `labels_train.jsonl` (533 rows). Each row is kept only if it has exactly one label per word.
- **Two losses are added together:** `tag_loss` grades every word's 17-way choice (padding, marked −100, is ignored), and `q_loss` grades the sentence's yes/no "add how" choice. One `backward()` improves both heads at once.
- It trains for 100 epochs with seed 42, and saves `trained_picker.pt` with `"variant": "picker"`.

C11. train_distilbert.py: training Restructure (not in the pipeline now)

It reads the 941 Restructure rows. `build_position_labels()` works out each input word's correct position (`sad mother` → `mother sad` gives labels `[1, 0]`). It then trains `DistilBertForRestructure` for 100 epochs and saves `trained_distilbert.pt`. It's kept in the folder in case your adviser brings Restructure back.

C12. slm_pipeline.py: the adviser's pipeline

Top of the file: the shared tables.

```
LOOKUP_TABLE = set(ALL_VOCAB)
SIGNS = set(CONFIRMED_GESTURES)
MAX_WORDS = 16
PEOPLE = {w for w, c in CATEGORIES.items() if c == "Person / Family"}
...
CONTRACTIONS = {"don't": "do not", ...}
PRONOUN_FITS = {"he": {"father", "teacher", "friend"}, "she": {"mother", "teacher", "friend"}}
COVER_FORMS = {"my": {"my", "i", "me", "am"}, "you": {"you", "your", "are"}}
```

These are the lookup table, the signs, the word groups used by Active Context, and the tables used by Normalization, Active Context and the safety net.

`class Turn` records one pass through a line: `steps` (the box name and what it produced, in order) and `output` (the screen text). The demos print these steps.

`class ActiveContext` is the notebook shared by both lines:

- `reset()` clears everything.
- `resolve_pronoun()` and `note_hearing()` serve Line 1; `fill_subject()` and `note_deaf()` serve Line 2 (all in Part B).
- `_remember_person()` stores the last person word seen.

`to_declarative(sentence)` (answer form):

```
words = sentence.split()
if len(words) > 2 and words[0] == "is":
    rest = words[1:]
    for i, w in enumerate(rest):
        if w in ("at", "home", "school"):
            return " ".join(rest[:i] + ["is"] + rest[i:])
    return " ".join(rest[:1] + ["is"] + rest[1:])
return sentence
```

If the sentence starts with `is`, move `is` to just before `at`, `home` or `school`, or else after the first word:

- `is mother home` → `mother is home`
- `is her mother at school` → `her mother is at school`

`covered(sign, out_words)` (safety net): a sign counts as present if its spelling, a synonym of it, a special form (`my` → `i` or `am`), or a word starting with its first 4 letters appears in the sentence.

`class SLMPipeline:`

- `__init__` builds the tokenizer, loads the two trained models with `_load()`, and starts an empty `ActiveContext`.
- `_load()` opens a `.pt` file, rebuilds the network from the settings saved inside it, loads the learned numbers, and calls `eval()` (switching dropout off).
- The box methods are covered one by one in Part B.

The two lines:

```
words, changes = self.normalization(text)
kept, removed = self.reduction(words)
ids = self.tokenize(kept)
vectors = self.embedding(ids, "speech")
hidden = self.contextualize(vectors)
glosses, labels, is_question = self.space_restriction(hidden, kept)
glosses, resolved = self.context.resolve_pronoun(words, glosses)
self.context.note_hearing(words, glosses)
out = self.transcribed_words(glosses)
self.display(turn, out)
```

That's `non_deaf_to_deaf()`, after Voice Recording and Speech to text. Each step is also recorded with `turn.add(...)` for the trace. `deaf_to_non_deaf()` does the same for Line 2: `transcribed_signs` → `tokenize` → `embedding(..., "signs")` → `fill_subject` (+ adding the person's vector) → `sentence_completion` → `note_deaf` → `display`.

Helpers at the bottom:

- `picker_training_words()` reads the picker's training words once and remembers them.
- `known_form()` gives the trained form of a synonym.
- `format_turn()` prints a turn box by box.
- `main()` runs the example conversation (`python slm_pipeline.py`).

C13. speech_input.py: microphone and speech-to-text

- `SAMPLE_RATE = 16000`, `RECORD_SECONDS = 5.0`, and `MODEL_DIR` (where the Vosk model must be unzipped).
- `SpeechUnavailable`: the error raised with a clear "how to install" message.
- `is_available()` tries to import `sounddevice` and `vosk`, and checks the model folder exists. The demo uses it to decide whether to offer the microphone option.
- `record()` and `transcribe()` are explained in Part B, Boxes 1–2.

Setup, once your group decides: `pip install vosk sounddevice`, then download `vosk-model-small-en-us-0.15.zip` (about 40 MB) from alphacephei.com/vosk/models and unzip it into `slm_training_v2/models/`.

C14. full_demo.py: the live demo with every box traced

```
slm = SLMPipeline()
mic = speech_input.is_available()
...
if choice == "1":
    turn = slm.deaf_to_non_deaf(text)
elif choice == "2":
    turn = slm.non_deaf_to_deaf(text)
elif choice == "3" and mic:
    turn = slm.non_deaf_to_deaf()          # records from the microphone
print("\n".join(format_turn(turn)))
screen(reader, turn.output or "(nothing)")
```

- It creates one pipeline (which loads both models), then loops on a menu:
  - `1` = the Deaf person signs
  - `2` = type what the hearing person says

- `3` = speak into the microphone (only offered when Vosk is installed)
- `r` = reset the memory (`slm.context.reset()`)
- `q` = quit

- After each turn it prints every box's output, draws the screen, and shows who the memory is holding. It contains **no pipeline logic of its own**: everything happens in `slm_pipeline.py`.

C15. two_way_demo.py: the compact demo

The same pipeline and the same menu (without the microphone option), but it prints only the two screens and the memory. Real run: `How is your father?` → `how father`, then signs `fine` → `father is fine`.

C16. Conversation_test.py: scripted conversations

```
for title, turns in CONVERSATIONS:
    for who, text in turns:
        slm.context.reset()
        if who == "sign":
            got = slm.deaf_to_non_deaf(text).output
            expected = ref.get(("complete", text))
        else:
            got = slm.non_deaf_to_deaf(text).output
            expected = ref.get(("reduce", text.lower()))
```

- `CONVERSATIONS` holds 7 scenes, 51 turns in total. `load_reference()` reads both dataset files into a dictionary of `(task, input) → answer`.
- For each turn it clears the memory (so each answer is judged on its own), runs the pipeline, and compares the result with the dataset's answer:
  - **MATCH** (same answer)
  - **DIFFER** (a real miss)
  - **NEW** (not in the dataset; you judge it)
- **Real results:** MATCH 45/51, DIFFER 0, NEW 6. The Deaf side uses 30 of the 40 signs.

C17. compare.py: Reduce accuracy on held-out sentences

```
pick_out, _, is_q = picker_predict(picker, tokenizer, inp, device)
if norm_text(pick_out) == norm_text(expected):
    pick_correct += 1
```

- For each of the 294 held-out rows in `labels_test.jsonl`, it runs the picker and compares the result with the answer. `norm_text` ignores capitals and punctuation.
- It also checks the question flag, and whether any output word was never spoken.
- **Real results:** 290/294 correct (98.6%), and the question flag is right on 294/294.
- The "decoder" column now shows 0/294, because TinyLlama is trained only for Sentence Completion. Don't report that column.

Part D: Glossary

Word	Meaning
Attention	Words exchange information, weighted by how relevant they are to each other
Backpropagation	Working out how to change every model number to reduce the loss
Batch	A small group of examples trained together (8 here)
Bidirectional	Every word can look at words before and after it (the encoder)
Causal	A word can only look at earlier words (the decoder)
Cross-attention	The sentence being written looks at the signs
Cross-entropy	The standard "how wrong" score when choosing one answer from several
Decoder	A model that writes one word at a time (TinyLlama)
Dropout	Randomly switching off numbers during training; off during use
Embedding	A learned table that turns an ID into 64 numbers
Encoder	A model that reads the whole input at once (the word picker's body)
Epoch	One full pass through the training data
Greedy decoding	Always taking the highest-scoring next word
Hungarian algorithm	Finds the best one-to-one match between words and seats
LayerNorm / RMSNorm	Rescale numbers to a steady range
Logits	Raw scores before they're turned into percentages
Loss	One number for how wrong the model is
Lookup table	The 88 words the SLM knows (`vocab_pool.ALL_VOCAB`)
Parameters	The learned numbers inside a model
Pre-trained	Trained by someone else; used as it is (Vosk)
Residual connection	Adding a layer's changes to its input instead of replacing it
RoPE	Rotary position embedding: position shown by rotating numbers
Softmax	Turns scores into percentages that add up to 100%
Teacher forcing	During training, the decoder is always given the correct previous words
Tensor, shape	A box of numbers, and how many there are in each direction
Token, ID	A word's position in the vocabulary table